



**Федеральное агентство по рыболовству**  
**Федеральное государственное бюджетное образовательное**  
**учреждение высшего образования**  
**«Астраханский государственный технический университет»**

Система менеджмента качества в области образования, воспитания, науки и инноваций  
ООО «ДКС РУС» по международному стандарту ISO 9001:2015

**ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОММУНИКАЦИЙ**  
**КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ**

**ЗАЩИТА ИНФОРМАЦИИ**

**ОТЧЕТ**

о выполнении лабораторной работы № 5  
Комплексное обеспечение безопасности менеджера учётных записей

Выполнил:  
студент гр. ЗИНРб-51

\_\_\_\_\_ Генералов В. В.

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

Преподаватель:

\_\_\_\_\_ Савоськин И. А.

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

**Астрахань – 2026**

## ВВЕДЕНИЕ

В работе разработан менеджер учётных записей, в котором совместно применяются хэширование паролей, контроль целостности базы данных (БД), шифрование файлов и журнал событий, поскольку ни один механизм защиты по отдельности не закрывает всех угроз.

Цель работы: «ознакомиться с принципами комплексной защиты информации на примере обеспечения защиты менеджера учетных записей».

Задание по методическим указаниям состоит из четырёх пунктов:

1. «Разработать программу, реализующую управление произвольными учетными записями. Программа должна иметь графический интерфейс, позволяющий просматривать, создавать, удалять и модифицировать имеющиеся учетные записи в файле БД. Пары имя пользователя + пароль должны храниться в отдельном текстовом файле, представляющем собой БД учетных записей. Хранение паролей должно осуществляться в хэшированном виде, т.е. при изменении/добавлении пароля в файл БД записывается его хэш-сумма (пароли не хранятся в открытом виде). Разрешено использовать сторонние функции и процедуры, реализующие алгоритм хэш-функции.
2. Добавить обеспечение целостности БД учетных записей путем проверки хэш-суммы. При каждом изменении БД посредством программы, в отдельный текстовый файл должна сохраняться хэш-сумма данных из БД. По запросу пользователя программа должна сверять актуальную хэш-сумму БД с эталонной, сохраненной в файле. В случае совпадения программа должна выдавать сообщение, что БД в целостности, при несовпадении – что БД была изменена извне.
3. Обеспечить криптографическую защиту файлов БД, журнала событий и хэш-суммы при помощи симметричной криптосистемы. Использовать отдельный ключ для каждого файла. Программа должна шифровать/расшифровывать файлы по желанию пользователя. Подсистема шифрования не должна конфликтовать с подсистемой обеспечения целостности. Разрешено использовать сторонние функции и процедуры, реализующие алгоритмы симметричного шифрования.
4. Добавить в программу журнал событий, представляющий собой отдельный текстовый файл, в который будут заноситься дата и время следующих операций: добавление, изменение, удаление учетных записей; шифрование/расшифрование файлов; сохранение/сравнение хэш-суммы файла БД».

Выполнение пунктов задания подтверждено снимками окон программы, содержимым файлов данных и журнала событий, а также автотестом всех подсистем.

## 1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Хранение паролей в открытом виде недопустимо: при компрометации БД злоумышленник получает прямой доступ к системе. Вместо пароля хранится его хэш; при проверке вычисляется хэш введённого пароля и сравнивается с сохранённым. Соль – случайная строка, добавляемая к паролю перед хэшированием: одинаковые пароли дают разные хэши, а заранее вычисленные (радужные) таблицы становятся бесполезными. Медленные функции формирования ключа из пароля – PBKDF2 (Password-Based Key Derivation Function 2), bcrypt, Argon2 – повторяют хэширование тысячи раз и во столько же раз удорожают перебор.

Целостность данных также контролируется хэш-функцией, например алгоритмом SHA-256 (Secure Hash Algorithm с 256-битным результатом): при сохранении вычисляется хэш-сумма, которая хранится отдельно; позднее она вычисляется заново и сравнивается с эталонной. Любое изменение данных меняет хэш, а подобрать иные данные с тем же хэшем практически невозможно, поэтому совпадение означает неизменность.

Конфиденциальность файлов обеспечивает симметричное шифрование: один ключ служит и для шифрования, и для расшифрования. Стандарт шифрования AES (Advanced Encryption Standard, федеральный стандарт США FIPS-197) шифрует блоки по 16 байт, выполняя от десяти до четырнадцати раундов подстановок и перестановок: AES-256 после начального AddRoundKey выполняет четырнадцать раундов SubBytes, ShiftRows, MixColumns (в последнем раунде опускается), AddRoundKey; S-блок вычисляется из арифметики поля Галуа  $GF(2^8)$ . Режим сцепления блоков шифртекста CBC (Cipher Block Chaining) связывает блоки через сложение по модулю 2 с предыдущим блоком шифртекста и случайный вектор инициализации (IV):  $C_i = E(D_i \oplus C_{i-1})$ ,  $C_0 = IV$ , где  $D_i$  и  $C_i$  – блоки данных и шифртекста,  $E$  – зашифрование блока. Поэтому одинаковые открытые тексты дают разные шифртексты. Перед шифрованием данные дополняются до длины, кратной 16 байтам, по схеме PKCS#7 (Public-Key Cryptography Standards #7). Ключ выводится из пароля функцией PBKDF2 со случайной солью, а код аутентификации сообщения на основе хэш-функции HMAC (Hash-based Message Authentication Code) подтверждает, что шифртекст не изменён и ключ верен.

Подсистема аудита фиксирует дату, время и содержание значимых действий, что позволяет выявлять подозрительную активность и расследовать инциденты.

## 2 ОПИСАНИЕ АЛГОРИТМА И ПРОГРАММЫ

### 2.1 Постановка задачи

Требования задания приведены во введении, алгоритмы и их параметры с обоснованием – в таблице 2.1. Где задание не задаёт конкретных средств, приняты следующие решения:

1. Функции PBKDF2-HMAC-SHA256, SHA-256 и HMAC-SHA256 взяты из модулей *hashlib* и *hmac* стандартной библиотеки Python, а шифр AES-256 в режиме CBC реализован самостоятельно по FIPS-197 (сторонние реализации разрешены, но не требуются) и проверен на векторах стандарта.
2. Строка БД – логин : соль : хэш, соль и хэш шестнадцатеричные; эталон обновляется при каждом изменении БД через программу и кнопкой «Сохранить хэш-сумму».
3. Хэш-сумма считается по открытому содержимому БД, шифрование обратимо байт в байт; пока файл зашифрован, операции с ним недоступны, а при зашифрованном журнале доступно только его расшифрование – так подсистемы не конфликтуют.
4. В журнал попадают также проверки паролей и отказы при неверном пароле файла БД или хэш-суммы (отказ по самому журналу записать некуда – он зашифрован).
5. Если файл БД удалён извне, а эталон остался, БД не создаётся заново молча: проверка сообщает об удалении, пустую БД создаёт кнопка «Добавить».
6. БД и эталон меняются только вместе: при изменении учётной записи сначала записывается БД, затем эталон и строка журнала; при сбое записи эталона или журнала БД и эталон возвращаются к прежнему содержимому.

Таблица 2.1 – Параметры реализации

| Параметр       | Значение                                                           | Обоснование                                                      |
|----------------|--------------------------------------------------------------------|------------------------------------------------------------------|
| Хэш пароля     | PBKDF2-HMAC-SHA256, 100 000 итераций, соль 16 байт                 | Разные хэши одинаковых паролей; перебор замедлен                 |
| Хэш-сумма БД   | SHA-256 файла <i>accounts.txt</i> , формат хэш <i>accounts.txt</i> | Стандартная функция, лавинный эффект                             |
| Шифр           | AES-256, режим CBC, дополнение PKCS#7, случайный IV                | FIPS-197; одинаковые файлы дают разные шифртексты                |
| Ключ файла     | PBKDF2 (пароль, соль    имя файла), 200 000 итераций, 64 байта     | Отдельный ключ для каждого файла; разделение ключей шифра и HMAC |
| Аутентификация | HMAC-SHA256 по соли, IV и шифртексту                               | Неверный пароль и повреждение обнаруживаются до расшифрования    |

Имя файла входит в соль ключа: ключи файлов различны даже при одинаковых паролях.

## 2.2 Описание алгоритма

Пароли. Для логина  $L$  и пароля  $P$  с солью  $s$  из 16 случайных байт вычисляется хэш  $H$ :

$$H = \text{PBKDF2}(\text{HMAC-SHA256}, P, s, 100\,000). \quad (2.1)$$

В БД записывается строка  $L:s:H$ . При проверке введённый пароль  $P'$  хэшируется по формуле (2.1) с сохранённой солью  $s$ , и результат сравнивается с  $H$  за постоянное время.

Целостность. Эталонная хэш-сумма (эталон)  $h = \text{SHA-256}(F)$ , где  $F$  – содержимое файла *accounts.txt*, записывается в *accounts.hash* после каждого изменения БД. По запросу вычисляется актуальная хэш-сумма  $h'$  и сравнивается с  $h$ : при равенстве выводится сообщение «БД в целостности», иначе – «БД была изменена извне».

Шифрование файла. Из пароля файла  $K$ , случайной соли файла  $s_f$  (16 байт) и имени файла  $N$  по формуле (2.2) выводятся 64 байта, которые делятся на ключ шифрования  $K_{\text{enc}}$  и ключ HMAC  $K_{\text{mac}}$  по 32 байта. Содержимое файла  $D$ , дополненное по схеме PKCS#7, шифруется AES-256-CBC со случайным вектором инициализации  $IV$  в шифртекст  $C$ , к которому добавляется код аутентификации  $T$ :

$$(K_{\text{enc}}, K_{\text{mac}}) = \text{PBKDF2}(\text{HMAC-SHA256}, K, s_f \parallel N, 200\,000), \quad (2.2)$$

$$C = \text{AES-256-CBC}_{K_{\text{enc}}}(IV, D), T = \text{HMAC-SHA256}_{K_{\text{mac}}}(s_f \parallel IV \parallel C).$$

Файл заменяется текстом: строка-признак и  $\text{base64}(s_f \parallel IV \parallel C \parallel T)$ . При расшифровании сначала пересчитывается  $T$ : несовпадение означает неверный пароль или повреждение файла, и расшифрование не выполняется.

## 2.3 Описание программы

Программа состоит из трёх модулей на Python 3 (только стандартная библиотека): *aes.py* – шифр AES-CBC с самопроверкой на векторах FIPS-197; *storage.py* – класс *Store* с логикой всех подсистем и автотестом; *manager.py* – окно tkinter. Запуск – команда «python3 lab5/manager.py», автотест без интерфейса – «python3 lab5/storage.py», проверка шифра – «python3 lab5/aes.py». Листинги модулей приведены в приложении 1.

Класс *Store* работает с файлами *accounts.txt* (БД), *accounts.hash* (эталон) и *events.log* (журнал) в папке *data* рядом с программой. Методы *add\_account*, *update\_account*, *delete\_account*, *verify\_password* ведут учётные записи (логин без пробелов, двоеточий и ведущего «#», пароль от 1 до 128 символов, дубликаты отклоняются); *save\_hash* и *check\_integrity* – целостность; *encrypt\_file* и *decrypt\_file* – шифрование по паролю файла; *log* – журнал.

Главное окно программы содержит четыре блока: «Учётные записи» – таблица (логин, соль, хэш) с кнопками «Добавить», «Изменить» (переименование и/или новый пароль), «Удалить», «Проверить пароль», «Обновить»; «Шифрование файлов» – состояние каждого

файла (например, «открыт, 411 байт» или «ЗАШИФРОВАН») и кнопки «Зашифровать» (пароль вводится дважды) и «Расшифровать»; «Целостность БД» – эталонная и актуальная хэш-суммы, кнопки «Сохранить хэш-сумму» и «Проверить целостность»; «Журнал событий» – содержимое файла **events.log**. Строка состояния внизу дублирует результат последней операции.

Вид главного окна приведён на рисунке 2.1.



Рисунок 2.1 – Главное окно программы

Кнопки «Добавить» и «Изменить» открывают диалог учётной записи, показанный на рисунке 2.2: пароль вводится дважды, при изменении пустое поле пароля оставляет прежний пароль.

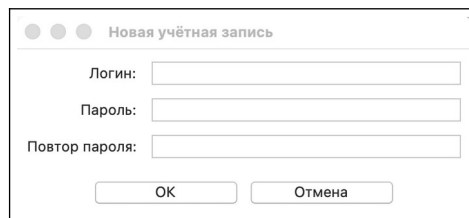


Рисунок 2.2 – Диалог добавления учётной записи

Изменение, удаление и проверка пароля относятся к выделенной строке, пароль файла вводится в диалоге «Ключ файла»; ошибки пользователя (зашифрованный или повреждённый файл, неверный пароль, дубликат, нет прав на запись) показываются диалоговым окном.

### 3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

#### 3.1 Учётные записи

Снимки окон и содержимое файлов получены в нескольких запусках программы с заново созданной БД, поэтому соли и хэш-суммы на разных рисунках могут различаться. В БД, показанной в окне программы (см. рис. 2.1), три учётные записи, пароль одной из них изменён; таблица показывает логин, соль и хэш – пароли не отображаются и не хранятся. Файлы БД и эталона приведены на рисунке 3.1: у каждой записи своя соль, паролей `qwerty123`, `NewPass#7`, `S3cret!` нет.

```
accounts.txt:
# логин:соль(hex):PBKDF2- HMAC-SHA256(пароль, соль, 100000 итераций)(hex)
ivan:4ed20390db88640df1f0995c30372c27:
    575e1d8c6c585d6d193f5cf60357d3aa77d37976a9318333b863497bdfc129e6
maria:62d4c3285bee3ec40d982530ab512712:
    691d7c473693bac2f677eb122032573063411ac7aae861451eb0e5b321fec31d
admin:db0f5d6d1dde6c632fd54420b27acdce:
    199fc3ab4b1a8f58e7172c18b5c88b52906ec33a1c27445cda0ca14cfe8dbf6a

accounts.hash:
b931004efa1b0536a427b693eded4a2d705635f38d725bda6aad77b170758274  accounts.txt
```

Рисунок 3.1 – Содержимое файлов **accounts.txt** и **accounts.hash**

Утечка такого файла не даёт входа в систему, а одинаковые пароли разных пользователей имеют разные хэши. Проверка пароля занимает около 12 мс (100 000 итераций PBKDF2) – примерно 85 паролей в секунду против 2,7 млн вызовов SHA-256 в секунду: словарь из миллиона паролей перебирается за 3,3 часа вместо доли секунды.

#### 3.2 Контроль целостности

Для нетронутой БД проверка сообщает «БД в целостности», как показано на рисунке 3.2 слева. После дописывания строки `hacker:...` в **accounts.txt** текстовым редактором проверка сообщает «БД была изменена извне» и выводит эталонную и актуальную хэш-суммы (рисунок 3.2, справа; части рисунка получены в разных запусках).

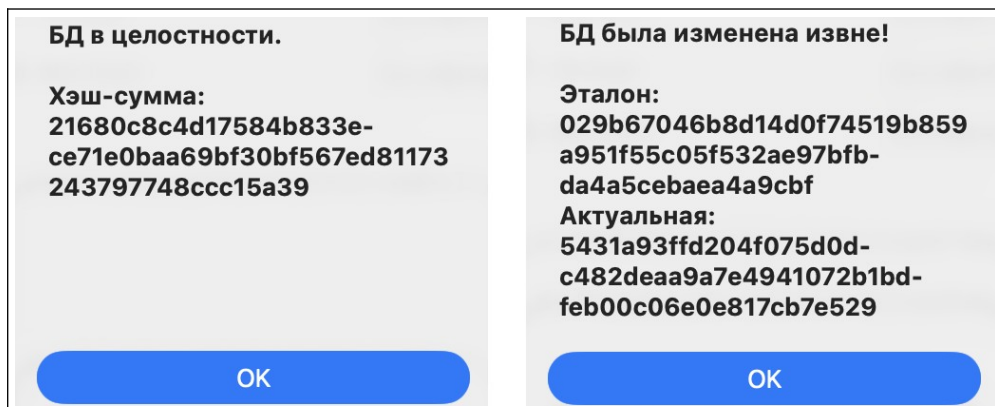


Рисунок 3.2 – Проверка целостности: слева – БД не изменена, справа – после правки **accounts.txt** извне

Дописанная извне строка изменила около половины бит хэш-суммы – лавинный эффект

SHA-256; подобрать правку с тем же хэшем практически невозможно, поэтому обнаруживается любое изменение файла. Ложных срабатываний нет: эталон обновляется вместе с БД при каждом изменении через программу, а удаление записи через программу или кнопка «Сохранить хэш-сумму» снова согласуют БД и эталон.

### 3.3 Шифрование файлов

Кнопка «Зашифровать» запрашивает пароль файла с повтором. После шифрования БД и эталона их состояние – «ЗАШИФРОВАН», список учётных записей – «(БД зашифрована)», поля хэш-сумм – «(файл зашифрован)», как видно на рисунке 3.3; операции с этими файлами блокируются (таблица 3.1).



Рисунок 3.3 – БД и файл хэш-суммы зашифрованы

Вид зашифрованного **accounts.hash** показан на рисунке 3.4.

```
accounts.hash после шифрования:
#ENCRYPTED AES-256-CBC, HMAC-SHA256, ключ: PBKDF2-HMAC-SHA256 из пароля файла
ZuxFN9YmEftp8dngmFn0r5zPoei8ksNqiNbY99ALciKwyGG05223XEoQIMLoq3GVG/7DU0heYs29
hPAbenQWvI33gZLZUMGSPyUw+EiR9I5MAf04JT5vA7bmSpkZjPmPiobiNkDqV2LM4c454Znen2eJ
Gc0LRZwfJEJZHh8T7LGK/3PcQzIqHjzgvVKgIOv8
```

Расшифрование с неверным паролем: неверный пароль или файл повреждён

Рисунок 3.4 – Зашифрованный **accounts.hash** и отказ при неверном пароле файла

Неверный пароль отклоняется до расшифрования, верный возвращает файл байт в байт, и проверка целостности снова сходится.



При зашифрованном журнале любая операция, кроме его расшифрования, отклоняется: на рисунке 3.5 показан отказ в добавлении записи.

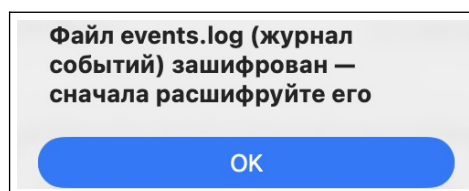


Рисунок 3.5 – Отказ в добавлении учётной записи при зашифрованном журнале

Зашифрованные файлы не содержат исходного текста; случайные соль и IV делают шифртексты одного файла различными при каждом шифровании. Скорость реализации AES – около 370 КБ/с – достаточна для текстовых файлов.

### 3.4 Журнал событий

Каждая строка журнала **events.log** содержит дату, время и описание события. В журнал заносятся все операции пункта 4 задания, а также проверки паролей учётных записей и отказы при неверном пароле файла; по таким отказам видна попытка подбора пароля к БД или эталону.

Журнал после сценария «добавление – изменение – проверка паролей – целостность – внешняя правка – удаление – сохранение эталона – шифрование, неверный пароль файла, расшифрование» приведён на рисунке 3.6.

|                     |                                                                                                                  |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| 2026-09-12 22:00:15 | создана пустая БД учётных записей, сохранена её хэш-сумма                                                        |
| 2026-09-12 22:00:15 | добавлена учётная запись «ivan»; хэш-сумма БД обновлена: 530604dc790e4a33...                                     |
| 2026-09-12 22:00:15 | добавлена учётная запись «maria»; хэш-сумма БД обновлена: 0d0b82eece3f0349...                                    |
| 2026-09-12 22:00:15 | добавлена учётная запись «admin»; хэш-сумма БД обновлена: 4745ac44fdc9109a...                                    |
| 2026-09-12 22:00:15 | изменена учётная запись «maria»: пароль изменён;<br>хэш-сумма БД обновлена: b931004efa1b0536...                  |
| 2026-09-12 22:00:15 | проверка пароля «ivan»: верный                                                                                   |
| 2026-09-12 22:00:15 | проверка пароля «admin»: НЕВЕРНЫЙ                                                                                |
| 2026-09-12 22:00:15 | проверка целостности БД: БД в целостности                                                                        |
| 2026-09-12 22:00:15 | проверка целостности БД: БД БЫЛА ИЗМЕНЕНА ИЗВНЕ (эталон b931004efa1b0536...,<br>актуальная a15b38fffb4786160...) |
| 2026-09-12 22:00:15 | удалена учётная запись «hacker»; хэш-сумма БД обновлена: b931004efa1b0536...                                     |
| 2026-09-12 22:00:15 | сохранена эталонная хэш-сумма БД: b931004efa1b0536...                                                            |
| 2026-09-12 22:00:15 | зашифрован файл: хэш-сумма БД (accounts.hash)                                                                    |
| 2026-09-12 22:00:15 | ОТКАЗ в расшифровании файла accounts.hash: неверный пароль или файл повреждён                                    |
| 2026-09-12 22:00:15 | расшифрован файл: хэш-сумма БД (accounts.hash)                                                                   |

Рисунок 3.6 – Журнал событий **events.log**

### 3.5 Анализ результатов

Результаты испытаний показывают, что подсистемы защиты дополняют друг друга и закрывают разные угрозы: хэширование паролей делает утечку файла БД бесполезной для входа в систему, контроль целостности обнаруживает правку БД в обход программы, шифрование скрывает содержимое файлов при хранении, а журнал позволяет восстановить ход событий и заметить попытки подбора паролей. Конфликтов между подсистемами не выявлено: после расшифрования проверка целостности сходится, эталон меняется только вместе с БД, а операции с зашифрованными файлами блокируются.

Автотест модуля **storage.py** повторяет сценарий во временной папке: все двадцать пять проверок пройдены (таблица 3.1). Самопроверка модуля **aes.py**: шифртексты AES-128, AES-192 и AES-256 совпадают с векторами FIPS-197, режим CBC обратим на данных от 0 до 1000 байт.

**Таблица 3.1 – Автоматическая проверка подсистем**

| Подсистема     | Проверяемое свойство                                                                                                                                        | Пройдено |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Учётные записи | Пароли не хранятся открыто; верный и неверный пароль различаются; смена пароля, переименование; дубликат и логин с «#» отклонены; своя соль у каждой записи | 9/9      |
| Целостность    | Совпадение после изменений через программу; внешняя правка обнаружена; эталон обновлён; испорченные файлы хэш-суммы и БД дают сообщение об ошибке           | 5/5      |
| Шифрование     | Три файла зашифрованы и нечитаемы; операции с ними блокируются; неверный пароль файла отвергнут; после расшифрования БД в целостности, записи сохранились   | 9/9      |
| Журнал         | Все виды событий присутствуют; дата и время в каждой строке                                                                                                 | 2/2      |

Пункты задания подтверждены: пункт 1 (см. рис. 2.1, 2.2, 3.1), пункт 2 (см. рис. 3.2), пункт 3 (см. рис. 3.3–3.5), пункт 4 (см. рис. 3.6), все подсистемы (см. таблицу 3.1).

#### 4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

**Вопрос 1.** Почему, как правило, пароли не хранятся в открытом виде? Как обеспечивать хранение паролей не в открытом виде, при этом имея возможность проверять их правильность?

БД с открытыми паролями при утечке или инсайдерском доступе даёт вход в систему, а пользователи часто повторяют пароли на других сервисах. Хэширование решает задачу благодаря детерминированности и односторонности хэш-функции: хранится хэш от соли, соединённой с паролем, при входе хэш пересчитывается с той же солью и сравнивается – правильный пароль даёт совпадение, а восстановить пароль из хэша нельзя (см. рис. 3.1). Соль исключает одинаковые хэши и предвычисленные таблицы, медленная функция (PBKDF2, bcrypt, Argon2) замедляет перебор.

**Вопрос 2.** Для чего нужна подсистема аудита с точки зрения безопасности?

Аудит фиксирует, кто, когда и что сделал в системе (см. рис. 3.6). Это позволяет обнаружить подозрительную активность (серию неверных паролей, доступ в нерабочее время), расследовать инцидент по последовательности событий, доказать факт действий и сдерживает нарушителей внутри организации. Журнал сам должен быть защищён от правки: в работе он шифруется при хранении (см. рис. 3.5), в промышленных системах передаётся на отдельный сервер или подписывается.

**Вопрос 3.** Каким образом с помощью хэш-функции возможна проверка целостности цифрового объекта?

Хэш-функция отображает объект в короткое значение фиксированной длины: любое изменение входа меняет значение, а подобрать другой объект с тем же хэшем практически невозможно (стойкость ко второму прообразу). Эталонная хэш-сумма вычисляется, когда объект заведомо цел, и хранится отдельно; позже хэш пересчитывается и сравнивается: совпадение означает, что объект не изменён (см. рис. 3.2). Эталон нужно защищать, иначе его подменят вместе с объектом.

**Вопрос 4.** Для чего используется криптографическая подсистема безопасности?

Она обеспечивает конфиденциальность: без ключа содержимое файлов нельзя прочитать даже при полном доступе к диску или резервным копиям (см. рис. 3.4). Дополнительно она даёт аутентичность шифртекста (НМАС), а с асимметричными алгоритмами – электронную подпись и распределение ключей. Отдельные ключи для разных файлов ограничивают ущерб при компрометации одного, а вывод ключа из пароля через PBKDF2 замедляет перебор. Шифрование защищает данные в хранении и при передаче, остальные угрозы закрывают хэширование паролей, контроль целостности и аудит.

## ЗАКЛЮЧЕНИЕ

Разработан менеджер учётных записей с графическим интерфейсом и четырьмя подсистемами защиты: пароли хранятся как солёные хэши PBKDF2-HMAC-SHA256; целостность БД контролируется эталонной хэш-суммой SHA-256, сохраняемой при каждом изменении и сверяемой по запросу с сообщениями «БД в целостности» и «БД была изменена извне»; файлы БД, эталона и журнала шифруются AES-256-CBC с отдельным ключом для каждого файла и кодом аутентификации HMAC; журнал фиксирует дату и время всех операций.

Пункты 1–4 задания выполнены и подтверждены экранами программы, содержимым файлов и автотестом из двадцати пяти проверок; реализация AES совпадает с векторами FIPS-197. Шифрование не конфликтует с целостностью: хэш считается по открытым данным, шифрование обратимо байт в байт, операции с зашифрованными файлами блокируются.

Ограничения учебной реализации – бесключевой эталон и скорость шифра на чистом Python. SHA-256 без ключа защищает от случайных и неквалифицированных правок, а злоумышленник с правом записи может пересчитать эталон или вернуть старую пару файлов. Шифрование файла эталона не даёт подделать сам зашифрованный эталон (без пароля файла HMAC не сойдётся), но не мешает заменить его открытым файлом с пересчитанной хэш-суммой: программа распознаёт шифрование только по строке-признаку и не помнит, что файл был зашифрован; от отката защиты тоже нет. В общем случае нужны секрет, недоступный атакующему (HMAC или электронная подпись эталона, как в лабораторной работе №4), носитель только для чтения и учёт версий эталона. В открытом виде журнал не защищён от правки и удаления, поэтому в промышленных системах его пишут на отдельный сервер или подписывают; здесь шифрование защищает его лишь при хранении, между сеансами работы.

## ПРИЛОЖЕНИЕ 1

## ЛИСТИНГ ПРОГРАММЫ

Приложение содержит листинги трёх модулей программы, состав которых описан в подразделе 2.3. Все модули используют только стандартную библиотеку Python 3; строки кода, не помещающиеся по ширине рамки, перенесены. Листинг модуля **storage.py** (подсистемы защиты) приведён на рисунке П1.1.

```
#!/usr/bin/env python3
"""Лабораторная работа №5. Хранилище менеджера учётных записей: подсистемы защиты.

Модуль без графического интерфейса (его даёт manager.py). Здесь реализованы:
1) хранение паролей учётных записей в виде солёного хэша PBKDF2- HMAC-SHA256 и их
   проверка пересчётом хэша с той же солью;
2) контроль целостности БД: при каждом изменении через программу хэш-сумма SHA-256
   файла БД записывается в отдельный файл; по запросу актуальная хэш-сумма сверяется
   с эталонной («БД в целостности» / «БД была изменена извне»);
3) шифрование файлов БД, хэш-суммы и журнала симметричным шифром AES-256-CBC (aes.py)
   с отдельным ключом для каждого файла: ключ выводится из пароля файла функцией
   PBKDF2
   со случайной солью, подлинность шифртекста подтверждает HMAC-SHA256 (неверный
   пароль
   обнаруживается до расшифрования);
4) журнал событий: дата и время каждой значимой операции в текстовом файле.

Шифрование не конфликтует с контролем целостности: хэш-сумма считается по открытому
содержимому БД, а шифрование обратимо байт в байт, поэтому после расшифрования проверка
сходится. Все операции регистрируются в журнале, поэтому пока журнал зашифрован,
доступно только его расшифрование.

Файлы (папка data рядом с программой): accounts.txt – БД, accounts.hash – эталонная
хэш-сумма, events.log – журнал. Только стандартная библиотека.
Запуск: python3 storage.py – автоматическая проверка всех подсистем без GUI.
"""

import base64
import datetime
import hashlib
import hmac
import os
import shutil
import sys
import tempfile

BASE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, BASE)
import aes # noqa: E402 (собственная реализация AES-CBC)

DATA_DIR = os.path.join(BASE, "data")
FILES = {"db": "accounts.txt", "hash": "accounts.hash", "log": "events.log"}
TITLES = {"db": "БД учётных записей", "hash": "хэш-сумма БД", "log": "журнал событий"}
ROLES = ("db", "hash", "log")

PASSWORD_ITERATIONS = 100_000 # PBKDF2 для паролей учётных записей
KDF_ITERATIONS = 200_000 # PBKDF2 для ключей шифрования файлов
MAGIC = "#ENCRYPTED AES-256-CBC, HMAC-SHA256, ключ: PBKDF2-HMAC-SHA256 из пароля файла"
DB_HEADER = "# логин:соль(hex):PBKDF2-HMAC-SHA256(пароль, соль, %d итераций)(hex)" %
    PASSWORD_ITERATIONS
LOGIN_MAX, PASSWORD_MAX = 32, 128

class ManagerError(Exception):
    """Ошибка, которую нужно показать пользователю (не сбой программы)."""
```

```

def first_upper(text):
    """Первая буква заглавная, остальное без изменений (capitalize() портит «БД»)."""
    return text[:1].upper() + text[1:]

# --- Пароли учётных записей
-----
def hash_password(password, salt=None):
    """Солёный хэш пароля. Возвращает (соль hex, хэш hex)."""
    salt = os.urandom(16) if salt is None else salt
    digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt,
        PASSWORD_ITERATIONS)
    return salt.hex(), digest.hex()

def check_password(password, salt_hex, hash_hex):
    """Пересчитывает хэш с сохранённой солью и сравнивает за постоянное время."""
    _, digest = hash_password(password, bytes.fromhex(salt_hex))
    return hmac.compare_digest(digest, hash_hex)

def check_login(login):
    if not login or len(login) > LOGIN_MAX:
        raise ManagerError(f"логин должен содержать от 1 до {LOGIN_MAX} символов")
    if any(ch.isspace() or ch == ":" or ord(ch) < 32 for ch in login):
        raise ManagerError("логин не может содержать пробелы, двоеточие и управляющие символы")
    if login.startswith("#"):
        raise ManagerError("логин не может начинаться с символа «#» (так помечены строки-комментарии БД)")

def is_hex(text, length):
    """Строка ровно из length шестнадцатеричных цифр (соль, хэш, эталон)."""
    return len(text) == length and all(ch in "0123456789abcdef" for ch in text)

def check_new_password(password):
    if not password or len(password) > PASSWORD_MAX:
        raise ManagerError(f"пароль должен содержать от 1 до {PASSWORD_MAX} символов")

# --- Шифрование файлов
-----
def derive_keys(password, salt, role):
    """Из пароля файла – ключ шифрования (32 байта) и ключ HMAC (32 байта).
    Имя файла входит в соль, поэтому у файлов разные ключи даже при одинаковом пароле."""
    material = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt +
        FILES[role].encode("utf-8"),
        KDF_ITERATIONS, dklen=64)
    return material[:32], material[32:]

def encrypt_data(data, password, role):
    """Открытые байты → текст: строка-признак и base64(соль | IV | шифртекст |
    HMAC)."""
    salt, iv = os.urandom(16), os.urandom(16)
    key_enc, key_mac = derive_keys(password, salt, role)
    body = salt + iv + aes.encrypt_cbc(key_enc, iv, data)
    tag = hmac.new(key_mac, body, hashlib.sha256).digest()
    b64 = base64.b64encode(body + tag).decode("ascii")
    lines = [b64[i:i + 76] for i in range(0, len(b64), 76)]
    return MAGIC + "\n" + "\n".join(lines) + "\n"

def decrypt_data(text, password, role):
    """Обратное преобразование. ManagerError при неверном пароле или повреждении файла."""
    lines = text.split("\n")

```

```

if not lines or lines[0] != MAGIC:
    raise ManagerError("файл не зашифрован этой программой")
try:
    raw = base64.b64decode("".join(lines[1:]), validate=True)
except (ValueError, TypeError):
    raise ManagerError("файл повреждён: не удалось прочитать base64")
if len(raw) < 16 + 16 + 16 + 32:
    raise ManagerError("файл повреждён: слишком короткий")
body, tag = raw[:-32], raw[-32:]
salt, iv, cipher = body[:16], body[16:32], body[32:]
key_enc, key_mac = derive_keys(password, salt, role)
if not hmac.compare_digest(hmac.new(key_mac, body, hashlib.sha256).digest(), tag):
    raise ManagerError("неверный пароль или файл повреждён")
try:
    return aes.decrypt_cbc(key_enc, iv, cipher)
except ValueError as exc:
    raise ManagerError(f"файл повреждён: {exc}")

# --- Хранилище: файлы, учётные записи, целостность, журнал
-----
class Store:
    def __init__(self, data_dir=DATA_DIR):
        self.dir = data_dir
        try:
            os.makedirs(self.dir, exist_ok=True)
        except OSError as exc:
            raise ManagerError(f"не удалось создать папку данных {self.dir}: {exc.strerror or exc}")
        if self.read("db") is None and self.read("hash") is None and not self.is_encrypted("log"):
            self.create_db() # первый запуск; если эталон есть, а БД нет — это удаление извне

    def create_db(self):
        """Пустая БД и её эталонная хэш-сумма (первый запуск или явное решение пользователя)."""
        self.require_plain("hash", "log")
        self.write("db", (DB_HEADER + "\n").encode("utf-8"))
        self.write("hash", self.hash_line())
        self.log("создана пустая БД учётных записей, сохранена её хэш-сумма")

    def path(self, role):
        return os.path.join(self.dir, FILES[role])

    def read(self, role):
        """Байты файла или None, если файла нет."""
        try:
            with open(self.path(role), "rb") as f:
                return f.read()
        except FileNotFoundError:
            return None
        except OSError as exc:
            raise ManagerError(f"не удалось прочитать файл {FILES[role]}: {exc.strerror or exc}")

    def write(self, role, data):
        """Запись через временный файл: на диске либо старое содержимое, либо новое целиком."""
        tmp = self.path(role) + ".tmp"
        try:
            with open(tmp, "wb") as f:
                f.write(data)
            os.replace(tmp, self.path(role))
        except OSError as exc:
            try:
                os.remove(tmp)
            except OSError:
                pass
            raise ManagerError(f"не удалось записать файл {FILES[role]}: {exc.strerror or exc}")

```

```

        or exc}")

def is_encrypted(self, role):
    data = self.read(role)
    return data is not None and data.startswith(MAGIC.encode("utf-8") + b"\n")

def require_plain(self, *roles):
    """Операция возможна, только если перечисленные файлы не зашифрованы."""
    for role in roles:
        if self.is_encrypted(role):
            raise ManagerError(f"файл {FILES[role]} ({TITLES[role]}) зашифрован – сначала расшифруйте его")

# --- журнал событий
def log(self, event):
    self.require_plain("log")
    stamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    try:
        with open(self.path("log"), "ab") as f:
            f.write(f"{stamp} {event}\n".encode("utf-8"))
    except OSError as exc:
        raise ManagerError(f"не удалось записать журнал {FILES['log']}: {exc.strerror or exc}")

def log_lines(self):
    data = self.read("log")
    if data is None:
        return []
    return data.decode("utf-8", "replace").splitlines()

# --- учётные записи
def load_accounts(self):
    """Список (логин, соль hex, хэш hex). ManagerError, если БД зашифрована или повреждена."""
    self.require_plain("db")
    data = self.read("db")
    if data is None:
        raise ManagerError(f"файл {FILES['db']} не найден")
    accounts = []
    for number, line in enumerate(data.decode("utf-8-sig", "replace").splitlines(), 1):
        if not line.strip() or line.startswith("#"):
            continue
        parts = line.split(":")
        if len(parts) != 3 or not parts[0] or not is_hex(parts[1], 32) or not is_hex(parts[2], 64):
            raise ManagerError(f"файл БД повреждён (строка {number})")
        accounts.append(tuple(parts))
    return accounts

def save_accounts(self, accounts, event):
    """Запись БД, обновление эталонной хэш-суммы и запись в журнал – одно изменение."""
    self.require_plain("db", "hash", "log")
    old_db, old_hash = self.read("db"), self.read("hash")
    lines = [DB_HEADER] + [":".join(acc) for acc in accounts]
    self.write("db", ("\n".join(lines) + "\n").encode("utf-8"))
    try:
        self.write("hash", self.hash_line())
        self.log(f"{event}; хэш-сумма БД обновлена: {self.db_hash()[:16]}...")
    except ManagerError:
        try:
            # откат: БД и эталон меняются только вместе
            if old_db is not None:
                self.write("db", old_db)
            if old_hash is not None:
                self.write("hash", old_hash)
        except ManagerError:
            pass
        raise

```



```

def add_account(self, login, password):
    check_login(login)
    check_new_password(password)
    accounts = self.load_accounts()
    if any(acc[0] == login for acc in accounts):
        raise ManagerError(f"учётная запись «{login}» уже существует")
    accounts.append((login,) + hash_password(password))
    self.save_accounts(accounts, f"добавлена учётная запись «{login}»")

def update_account(self, login, new_login, new_password):
    """Переименование и/или смена пароля; пустой new_password – пароль не
        меняется."""
    check_login(new_login)
    if new_password:
        check_new_password(new_password)
    accounts = self.load_accounts()
    index = next((i for i, acc in enumerate(accounts) if acc[0] == login), None)
    if index is None:
        raise ManagerError(f"учётная запись «{login}» не найдена")
    if new_login != login and any(acc[0] == new_login for acc in accounts):
        raise ManagerError(f"учётная запись «{new_login}» уже существует")
    changes = []
    if new_login != login:
        changes.append(f"логин «{login}» → «{new_login}»")
    if new_password:
        changes.append("пароль изменён")
    if not changes:
        raise ManagerError("изменений нет")
    salt, digest = hash_password(new_password) if new_password else
        accounts[index][1:]
    accounts[index] = (new_login, salt, digest)
    self.save_accounts(accounts, f"изменена учётная запись «{login}»: " + ",
        ".join(changes))

def delete_account(self, login):
    accounts = self.load_accounts()
    rest = [acc for acc in accounts if acc[0] != login]
    if len(rest) == len(accounts):
        raise ManagerError(f"учётная запись «{login}» не найдена")
    self.save_accounts(rest, f"удалена учётная запись «{login}»")

def verify_password(self, login, password):
    """Проверка пароля по хэшу; результат заносится в журнал (аудит входов)."""
    self.require_plain("log")
    for acc in self.load_accounts():
        if acc[0] == login:
            ok = check_password(password, acc[1], acc[2])
            self.log(f"проверка пароля «{login}»: {'верный' if ok else
                'НЕВЕРНЫЙ'}")
            return ok
    raise ManagerError(f"учётная запись «{login}» не найдена")

# --- целостность БД
def db_hash(self):
    self.require_plain("db")
    data = self.read("db")
    if data is None:
        raise ManagerError(f"файл {FILES['db']} не найден: БД была удалена извне")
    return hashlib.sha256(data).hexdigest()

def hash_line(self):
    return f"{self.db_hash()} {FILES['db']}\n".encode("utf-8")

def stored_hash(self):
    """Эталон из файла хэш-суммы; None, если файла нет; ManagerError, если он
        повреждён."""
    self.require_plain("hash")
    data = self.read("hash")
    if data is None:

```

```

        return None
    tokens = data.decode("utf-8-sig", "replace").split()
    if not tokens or not is_hex(tokens[0], 64):
        raise ManagerError(f"файл {FILES['hash']} повреждён: не содержит хэш-суммы SHA-256")
    return tokens[0]

def save_hash(self):
    self.require_plain("db", "hash", "log")
    self.write("hash", self.hash_line())
    self.log(f"сохранена эталонная хэш-сумма БД: {self.db_hash()[0:16]}...")

def check_integrity(self):
    """Возвращает (совпало, эталон, актуальная). ManagerError, если эталона нет."""
    self.require_plain("db", "hash", "log")
    expected, actual = self.stored_hash(), self.db_hash()
    if expected is None:
        raise ManagerError("эталонная хэш-сумма не сохранена")
    ok = hmac.compare_digest(expected, actual)
    self.log("проверка целостности БД: " + ("БД в целостности" if ok
        else f"БД БЫЛА ИЗМЕНЕНА ИЗВНЕ (эталон {expected[:16]}..., актуальная {actual[:16]}...)"))
    return ok, expected, actual

# --- шифрование файлов
def encrypt_file(self, role, password):
    if not password:
        raise ManagerError("пароль файла не может быть пустым")
    if self.is_encrypted(role):
        raise ManagerError(f"файл {FILES[role]} уже зашифрован")
    if role != "log":
        self.require_plain("log")
    data = self.read(role)
    if data is None:
        raise ManagerError(f"файл {FILES[role]} не найден")
    event = f"зашифрован файл: {TITLES[role]} ({FILES[role]})"
    if role == "log":
        self.log(event) # последняя запись перед шифрованием
        самого журнала
        data = self.read(role)
    try:
        self.write(role, encrypt_data(data, password, role).encode("utf-8"))
    except ManagerError as exc:
        if role == "log":
            self.log(f"ОТКАЗ в шифровании файла {FILES[role]}: {exc}")
            raise
    if role != "log":
        self.log(event)

def decrypt_file(self, role, password):
    if not self.is_encrypted(role):
        raise ManagerError(f"файл {FILES[role]} не зашифрован")
    if role != "log":
        self.require_plain("log")
    text = self.read(role).decode("utf-8", "replace")
    try:
        plain = decrypt_data(text, password, role)
    except ManagerError as exc:
        if role != "log":
            self.log(f"ОТКАЗ в расшифровании файла {FILES[role]}: {exc}")
            raise
    self.write(role, plain)
    self.log(f"расшифрован файл: {TITLES[role]} ({FILES[role]})")

# --- Автоматическая проверка без GUI
-----
def self_test(verbose=True):
    """Сценарий из задания: учётные записи → целостность → внешнее изменение → шифрование."""

```

```

results = []

def step(name, ok):
    results.append(ok)
    if verbose:
        print(f" [{ 'ок' if ok else 'ОШИБКА' }] {name}")

tmp = tempfile.mkdtemp(prefix="lab5_")
try:
    st = Store(tmp)
    st.add_account("ivan", "qwerty123")
    st.add_account("maria", "Пароль!2026")
    db = st.read("db").decode("utf-8")
    step("пароли не хранятся открыто", "qwerty123" not in db and "Пароль!2026" not
        in db)
    step("два пользователя в БД", [a[0] for a in st.load_accounts()] == ["ivan",
        "maria"])
    step("верный пароль принимается", st.verify_password("ivan", "qwerty123"))
    step("неверный пароль отклоняется", not st.verify_password("ivan",
        "qwerty124"))
    st.update_account("ivan", "ivan", "newpass")
    step("смена пароля", st.verify_password("ivan", "newpass") and not
        st.verify_password("ivan", "qwerty123"))
    st.update_account("maria", "maria.p", "")
    step("переименование без смены пароля", st.verify_password("maria.p",
        "Пароль!2026"))
    try:
        st.add_account("ivan", "x")
        step("дубликат логина отклонён", False)
    except ManagerError:
        step("дубликат логина отклонён", True)
    salts = {a[1] for a in st.load_accounts()}
    step("у каждой записи своя соль", len(salts) == 2)

    ok, _, _ = st.check_integrity()
    step("целостность после изменений через программу", ok)
    with open(st.path("db"), "ab") as f: # изменение «извне»,
        # минуя программу
        f.write(("hacker:" + "00" * 16 + ":" + "00" * 32 + "\n").encode("ascii"))
    ok, _, _ = st.check_integrity()
    step("внешнее изменение обнаружено", not ok)
    st.delete_account("hacker")
    step("после изменения через программу эталон обновлён", st.check_integrity(
        )[0])
    good_hash = st.read("hash")
    st.write("hash", "мусор вместо хэша\n".encode("utf-8")) # порча файла
    # эталона «извне»
    try:
        st.check_integrity()
        step("испорченный файл хэш-суммы → сообщение об ошибке", False)
    except ManagerError:
        step("испорченный файл хэш-суммы → сообщение об ошибке", True)
    st.write("hash", good_hash)
    good_db = st.read("db")
    st.write("db", good_db + b"eve:not-hex-salt:not-hex-hash\n") # порча
    # строки БД «извне»
    try:
        st.load_accounts()
        step("испорченная строка БД → сообщение об ошибке", False)
    except ManagerError:
        step("испорченная строка БД → сообщение об ошибке", True)
    st.write("db", good_db)
    try:
        st.add_account("#root", "x")
        step("логин с «#» отклонён", False)
    except ManagerError:
        step("логин с «#» отклонён", True)

passwords = {"db": "key-db", "hash": "key-hash", "log": "key-log"}
secrets = {"db": "ivan:", "hash": st.stored_hash(), "log": "проверка

```

```

        целостности"} # что не должно читаться
for role in ROLES:
    st.encrypt_file(role, passwords[role])
step("все три файла зашифрованы", all(st.is_encrypted(r) for r in ROLES))
for role in ROLES:
    text = st.read(role).decode("utf-8", "replace")
    step(f"{FILES[role]}: содержимое не читается", secrets[role] not in text)
try:
    st.add_account("petr", "1")
    step("операции с зашифрованными файлами блокируются", False)
except ManagerError:
    step("операции с зашифрованными файлами блокируются", True)
try:
    st.decrypt_file("log", "wrong")
    step("неверный пароль файла отвергнут", False)
except ManagerError:
    step("неверный пароль файла отвергнут", True)
st.decrypt_file("log", passwords["log"])
try:
    st.decrypt_file("db", "wrong")
    step("неверный пароль БД отвергнут", False)
except ManagerError:
    step("неверный пароль БД отвергнут", True)
st.decrypt_file("db", passwords["db"])
st.decrypt_file("hash", passwords["hash"])
step("после расшифрования БД в целостности", st.check_integrity()[0])
step("учётные записи сохранились", [a[0] for a in st.load_accounts()] ==
      ["ivan", "maria.p"])

wanted = ["добавлена", "изменена", "удалена", "зашифрован файл", "расшифрован
          файл",
          "сохранена эталонная", "проверка целостности", "ОТКАЗ"]
st.save_hash()
lines = st.log_lines()
step("в журнале есть все виды событий", all(any(w in line for line in lines)
      for w in wanted))
step("записи журнала с датой и временем", all(len(line) > 20 and line[4] == "-"
      and line[10] == " " for line in lines))
if verbose:
    print("\nЖурнал событий:")
    for line in lines:
        print(" " + line)
finally:
    shutil.rmtree(tmp, ignore_errors=True)
total, good = len(results), sum(results)
if verbose:
    print(f"\nИтого: {good}/{total} проверок пройдено")
return good == total

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(errors="replace") # консоль Windows в cp866/cp1251
        не знает «↔» и «...»
    sys.exit(0 if self_test() else 1)

```

Рисунок П1.1 – Листинг модуля *storage.py*

Листинг модуля *manager.py* (графический интерфейс) приведён на рисунке П1.2.

```
#!/usr/bin/env python3
"""Лабораторная работа №5. Комплексное обеспечение безопасности менеджера учётных
записей.
```

Графический интерфейс (tkinter) к хранилищу storage.py: просмотр, добавление, изменение и удаление учётных записей в текстовом файле БД (пароли – только в виде солёного хэша), проверка целостности БД по эталонной хэш-сумме, шифрование и расшифрование файлов БД, хэш-суммы и журнала (AES-256-CBC, отдельный ключ для каждого файла) и журнал событий с датой и временем операций.

```

Только стандартная библиотека. Запуск: python3 manager.py [папка данных]
Автоматическая проверка подсистем без GUI: python3 storage.py
"""

import os
import sys
import tkinter as tk
from tkinter import ttk, messagebox, simplifiedialog

BASE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, BASE)
from storage import (DATA_DIR, FILES, ROLES, TITLES, ManagerError, Store, # noqa:
E402
                    check_login, check_new_password, first_upper)

class Dialog(simplifiedialog.Dialog):
    """Модальное окно с русскими кнопками (simplifiedialog.Dialog даёт OK/Cancel)."""

    def buttonbox(self):
        box = ttk.Frame(self)
        ttk.Button(box, text="OK", width=10, command=self.ok, default="active").pack(
            side="left", padx=5, pady=5)
        ttk.Button(box, text="Отмена", width=10, command=self.cancel).pack(side="left",
            padx=5, pady=5)
        self.bind("<Return>", self.ok)
        self.bind("<Escape>", self.cancel)
        box.pack()

class PasswordDialog(Dialog):
    """Ввод пароля (ключа файла или пароля пользователя); confirm=True – с полем
    повтора."""

    def __init__(self, parent, title, prompt, confirm=False):
        self.prompt, self.confirm, self.result = prompt, confirm, None
        super().__init__(parent, title)

    def body(self, master):
        ttk.Label(master, text=self.prompt).grid(row=0, column=0, columnspan=2,
            sticky="w", padx=4, pady=(4, 6))
        ttk.Label(master, text="Пароль:").grid(row=1, column=0, sticky="e", padx=4,
            pady=3)
        self.e_pass = ttk.Entry(master, width=30, show="•")
        self.e_pass.grid(row=1, column=1, padx=4, pady=3)
        if self.confirm:
            ttk.Label(master, text="Повтор пароля:").grid(row=2, column=0, sticky="e",
                padx=4, pady=3)
            self.e_pass2 = ttk.Entry(master, width=30, show="•")
            self.e_pass2.grid(row=2, column=1, padx=4, pady=3)
        return self.e_pass

    def validate(self):
        if not self.e_pass.get():
            messagebox.showerror("Ошибка", "Пароль не может быть пустым.", parent=self)
            return False
        if self.confirm and self.e_pass.get() != self.e_pass2.get():
            messagebox.showerror("Ошибка", "Пароли не совпадают.", parent=self)
            return False
        return True

    def apply(self):
        self.result = self.e_pass.get()

class AccountDialog(Dialog):
    """Окно добавления/изменения учётной записи."""

    def __init__(self, parent, title, login="", editing=False):

```

```

        self.login, self.editing, self.result = login, editing, None
        super().__init__(parent, title)

    def body(self, master):
        hint = "Новый пароль (пусто – не менять)" if self.editing else "Пароль"
        ttk.Label(master, text="Логин:").grid(row=0, column=0, sticky="e", padx=4,
            pady=3)
        ttk.Label(master, text=hint + ":").grid(row=1, column=0, sticky="e", padx=4,
            pady=3)
        ttk.Label(master, text="Повтор пароля:").grid(row=2, column=0, sticky="e",
            padx=4, pady=3)
        self.e_login = ttk.Entry(master, width=30)
        self.e_pass = ttk.Entry(master, width=30, show="•")
        self.e_pass2 = ttk.Entry(master, width=30, show="•")
        self.e_login.insert(0, self.login)
        for row, entry in enumerate((self.e_login, self.e_pass, self.e_pass2)):
            entry.grid(row=row, column=1, padx=4, pady=3)
        return self.e_login

    def validate(self):
        try:
            check_login(self.e_login.get())
            if self.e_pass.get() or not self.editing:
                check_new_password(self.e_pass.get())
            if self.e_pass.get() != self.e_pass2.get():
                raise ManagerError("пароли не совпадают")
        except ManagerError as exc:
            messagebox.showerror("Ошибка", first_upper(str(exc)), parent=self)
            return False
        return True

    def apply(self):
        self.result = (self.e_login.get(), self.e_pass.get())

class App(tk.Tk):
    """Главное окно: список учётных записей, шифрование файлов, целостность БД,
        журнал."""

    def __init__(self, store):
        super().__init__()
        self.store = store
        self.title("Менеджер учётных записей – лабораторная работа №5")
        self.minsize(820, 640)
        self.state_labels, self.hash_vars = {}, {}
        self.status = tk.StringVar(value=f"Папка данных: {store.dir}")
        self.build()
        self.refresh()

    # --- построение окна
    def build(self):
        pad = {"padx": 6, "pady": 4}
        frame_acc = ttk.LabelFrame(self, text="Учётные записи (accounts.txt)")
        frame_acc.pack(fill="both", expand=True, **pad)
        columns = ("login", "salt", "hash")
        self.tree = ttk.Treeview(frame_acc, columns=columns, show="headings", height=8)
        for col, title, width in (("login", "Логин", 200), ("salt", "Соль (hex)", 230),
            ("hash", "Хэш пароля PBKDF2-SHA256 (hex)", 360)):
            self.tree.heading(col, text=title)
            self.tree.column(col, width=width, anchor="w")
        scroll = ttk.Scrollbar(frame_acc, orient="vertical", command=self.tree.yview)
        self.tree.configure(yscrollcommand=scroll.set)
        self.tree.pack(side="left", fill="both", expand=True, padx=(6, 0), pady=4)
        scroll.pack(side="left", fill="y", pady=4)
        buttons = ttk.Frame(frame_acc)
        buttons.pack(side="left", fill="y", padx=6, pady=4)
        for text, cmd in (("Добавить", self.add), ("Изменить", self.edit), ("Удалить",
            self.delete),
            ("Проверить пароль", self.verify), ("Обновить",
            self.refresh)):

```

```

        ttk.Button(buttons, text=text, command=cmd, width=18).pack(fill="x",
            pady=2)

    frame_files = ttk.LabelFrame(self, text="Шифрование файлов (AES-256-CBC,
        отдельный ключ для каждого файла)")
    frame_files.pack(fill="x", **pad)
    for row, role in enumerate(ROLES):
        title = f"{TITLES[role][0].upper()} {TITLES[role][1]} – {FILES[role]}:"
        ttk.Label(frame_files, text=title, width=36).grid(row=row, column=0,
            sticky="w", padx=6, pady=3)
        self.state_labels[role] = ttk.Label(frame_files, text="", width=22)
        self.state_labels[role].grid(row=row, column=1, sticky="w", padx=6)
        ttk.Button(frame_files, text="Зашифровать", command=lambda r=role:
            self.encrypt(r)).grid(row=row, column=2, padx=3, pady=2)
        ttk.Button(frame_files, text="Расшифровать", command=lambda r=role:
            self.decrypt(r)).grid(row=row, column=3, padx=3, pady=2)

    frame_int = ttk.LabelFrame(self, text="Целостность БД (SHA-256, эталон в
        accounts.hash)")
    frame_int.pack(fill="x", **pad)
    for row, (key, title) in enumerate(("stored", "Эталонная хэш-сумма:"), (
        "actual", "Актуальная хэш-сумма:")):
        ttk.Label(frame_int, text=title, width=22).grid(row=row, column=0,
            sticky="w", padx=6, pady=2)
        self.hash_vars[key] = tk.StringVar()
        ttk.Label(frame_int, textvariable=self.hash_vars[key], font=("Courier",
            11)).grid(row=row, column=1, sticky="w", padx=6)
        ttk.Button(frame_int, text="Сохранить хэш-сумму", command=self.save_hash).grid(
            row=0, column=2, padx=6, pady=2, sticky="e")
        ttk.Button(frame_int, text="Проверить целостность", command=self.check).grid(
            row=1, column=2, padx=6, pady=2, sticky="e")
    frame_int.columnconfigure(1, weight=1)

    frame_log = ttk.LabelFrame(self, text="Журнал событий (events.log)")
    frame_log.pack(fill="both", expand=True, **pad)
    self.log_text = tk.Text(frame_log, height=9, font=("Courier", 11),
        state="disabled", wrap="none")
    log_y = ttk.Scrollbar(frame_log, orient="vertical",
        command=self.log_text.yview)
    log_x = ttk.Scrollbar(frame_log, orient="horizontal",
        command=self.log_text.xview)
    self.log_text.configure(yscrollcommand=log_y.set, xscrollcommand=log_x.set)
    self.log_text.grid(row=0, column=0, sticky="nsew", padx=(6, 0), pady=(4, 0))
    log_y.grid(row=0, column=1, sticky="ns", pady=(4, 0))
    log_x.grid(row=1, column=0, sticky="ew", padx=(6, 0), pady=(0, 4))
    frame_log.rowconfigure(0, weight=1)
    frame_log.columnconfigure(0, weight=1)

    ttk.Label(self, textvariable=self.status, anchor="w", relief="sunken").pack(
        fill="x", padx=6, pady=(0, 6))

# --- обновление состояния
def refresh(self, select=None):
    """Перечитывает файлы с диска; select – логин, который нужно выделить в
        списке."""
    for role in ROLES:
        try:
            data = self.store.read(role)
            if data is None:
                text = "файл отсутствует"
            elif self.store.is_encrypted(role):
                text = "ЗАШИФРОВАН"
            else:
                text = f"открыт, {len(data)} байт"
        except ManagerError as exc:
            text = f"не удалось прочитать: {exc}"
        self.state_labels[role].configure(text=text)
    focused = self.tree.focus()
    selected = select or (self.tree.item(focused, "values")[0] if focused else
        None) # сохраняем выбор

```

```

self.tree.delete(*self.tree.get_children())
try:
    for login, salt, digest in self.store.load_accounts():
        item = self.tree.insert("", "end", values=(login, salt, digest))
        if login == selected:
            self.tree.focus(item)
            self.tree.selection_set(item)
except ManagerError as exc:
    short = "(БД зашифрована)" if self.state_labels["db"].cget("text") ==
        "ЗАШИФРОВАН" else f"({exc})"
    self.tree.insert("", "end", values=(short, "", ""))
for key, role, getter in (("stored", "hash", self.store.stored_hash), (
    "actual", "db", self.store.db_hash)):
    try:
        self.hash_vars[key].set(getter() or "- не сохранена -")
    except ManagerError as exc:
        self.hash_vars[key].set("(файл зашифрован)" if
            self.state_labels[role].cget("text") == "ЗАШИФРОВАН" else f"({exc})")

try:
    text = "(журнал зашифрован)" if self.store.is_encrypted("log") else
        "\n".join(self.store.log_lines())
except ManagerError as exc:
    text = f"(не удалось прочитать журнал: {exc})"
self.log_text.configure(state="normal")
self.log_text.delete("1.0", "end")
self.log_text.insert("end", text)
self.log_text.see("end")
self.log_text.xview_moveto(0) # последние записи видны с начала
    строки
self.log_text.configure(state="disabled")

def selected_login(self):
    item = self.tree.focus()
    if not item or not self.tree.item(item, "values")[1]:
        raise ManagerError("выберите учётную запись в списке")
    return self.tree.item(item, "values")[0]

def guarded(self, action, done=None, select=None):
    """Выполняет действие, показывает ошибки и обновляет окно (select – логин для
        выделения)."""
    try:
        result = action()
    except (ManagerError, OSError) as exc:
        messagebox.showerror("Ошибка", first_upper(str(exc)), parent=self)
        self.status.set(f"Ошибка: {exc}")
        self.refresh()
        return
    self.refresh(select)
    if done:
        self.status.set(done if isinstance(done, str) else done(result))
    return result

def ask_password(self, role, verb, confirm=False):
    return PasswordDialog(self, "Ключ файла", f"{verb} файл {FILES[role]} (
        {TITLES[role]}).", confirm).result

# --- команды
def add(self):
    try:
        missing = self.store.read("db") is None
    except ManagerError:
        missing = False
    if missing:
        if messagebox.askyesno("БД отсутствует", f"Файл {FILES['db']} не найден.
            Создать пустую БД и сохранить её хэш-сумму?", parent=self):
            self.guarded(self.store.create_db, "Создана пустая БД")
        return
    dlg = AccountDialog(self, "Новая учётная запись")
    if dlg.result:

```



```

        login, password = dlg.result
        self.guarded(lambda: self.store.add_account(login, password), f"Добавлена
            учётная запись «{login}»")

def edit(self):
    try:
        login = self.selected_login()
    except ManagerError as exc:
        return messagebox.showinfo("Изменение", first_upper(str(exc)), parent=self)
    dlg = AccountDialog(self, f"Изменение учётной записи «{login}»", login,
        editing=True)
    if dlg.result:
        new_login, new_password = dlg.result
        self.guarded(lambda: self.store.update_account(login, new_login,
            new_password),
            f"Изменена учётная запись «{login}»", select=new_login)

def delete(self):
    try:
        login = self.selected_login()
    except ManagerError as exc:
        return messagebox.showinfo("Удаление", first_upper(str(exc)), parent=self)
    if messagebox.askyesno("Удаление", f"Удалить учётную запись «{login}»? ",
        parent=self):
        self.guarded(lambda: self.store.delete_account(login), f"Удалена учётная
            запись «{login}»")

def verify(self):
    try:
        login = self.selected_login()
    except ManagerError as exc:
        return messagebox.showinfo("Проверка пароля", first_upper(str(exc)),
            parent=self)
    password = PasswordDialog(self, "Проверка пароля", f"Введите пароль
        пользователя «{login}».").result
    if password is None:
        return
    ok = self.guarded(lambda: self.store.verify_password(login, password))
    if ok is not None:
        (messagebox.showinfo if ok else messagebox.showwarning)(
            "Проверка пароля", f"Пароль пользователя «{login}» " + ("верный." if ok
                else "НЕВЕРНЫЙ."), parent=self)
        self.status.set(f"Проверка пароля «{login}»: " + ("верный" if ok else
            "неверный"))

def save_hash(self):
    self.guarded(self.store.save_hash, "Эталонная хэш-сумма БД сохранена")

def check(self):
    result = self.guarded(self.store.check_integrity)
    if result is None:
        return
    ok, expected, actual = result
    if ok:
        messagebox.showinfo("Целостность", f"БД в целостности.\n\nХэш-сумма:
            {actual}", parent=self)
    else:
        messagebox.showwarning("Целостность", f"БД была изменена извне!\n\nЭталон:
            {expected}\n\nАктуальная: {actual}", parent=self)
    self.status.set("Проверка целостности: " + ("БД в целостности" if ok else "БД
        была изменена извне"))

def encrypt(self, role):
    password = self.ask_password(role, "Зашифровать", confirm=True)
    if password is not None:
        self.guarded(lambda: self.store.encrypt_file(role, password), f"Файл
            {FILES[role]} зашифрован")

def decrypt(self, role):
    password = self.ask_password(role, "Расшифровать")

```

```

        if password is None:
            return
        self.guarded(lambda: self.store.decrypt_file(role, password), f"Файл
            {FILES[role]} расшифрован")

def main(argv):
    data_dir = next((a for a in argv[1:] if not a.startswith("-")), DATA_DIR)
    try:
        app = App(Store(data_dir))
    except ManagerError as exc:
        tk.Tk().withdraw()
        messagebox.showerror("Менеджер учётных записей", first_upper(str(exc)))
        sys.exit(1)
    app.mainloop()

if __name__ == "__main__":
    main(sys.argv)

```

Рисунок П1.2 – Листинг модуля *manager.py*

Листинг модуля *aes.py* (шифр AES-CBC) приведён на рисунке П1.3.

```

#!/usr/bin/env python3
"""Лабораторная работа №5. Симметричный шифр AES (FIPS-197), режим CBC, дополнение
    PKCS#7.

Собственная реализация на стандартной библиотеке: S-блок вычисляется из арифметики поля
GF(2^8), поддерживаются ключи 128, 192 и 256 бит. Байты блока хранятся в порядке
    FIPS-197:
элемент состояния s[r][c] лежит в позиции r + 4с. Правильность проверяется на тестовых
векторах приложения С FIPS-197: python3 aes.py
"""

import os
import sys

BLOCK = 16          # размер блока в байтах

# --- Арифметика GF(2^8) и таблицы
-----
def _xtime(a):
    """Умножение на x в GF(2^8) с многочленом x^8 + x^4 + x^3 + x + 1."""
    a <= 1
    return (a ^ 0x1B) & 0xFF if a & 0x100 else a

def gmul(a, b):
    """Произведение элементов GF(2^8)."""
    p = 0
    while b:
        if b & 1:
            p ^= a
        a = _xtime(a)
        b >>= 1
    return p

def _make_sbox():
    """S-блок: обратный элемент в GF(2^8), затем аффинное преобразование (FIPS-197,
        5.1.1)."""
    inverse = [0] * 256
    for a in range(1, 256):
        if inverse[a]:
            continue
        for b in range(1, 256):
            if gmul(a, b) == 1:

```

```

        inverse[a], inverse[b] = b, a
        break
sbox = []
for x in range(256):
    a = inverse[x]
    s = a
    for i in range(1, 5):
        s ^= ((a << i) | (a >> (8 - i))) & 0xFF      # циклические сдвиги на 1..4
    sbox.append(s ^ 0x63)
return sbox

SBOX = _make_sbox()
INV_SBOX = [0] * 256
for _i, _s in enumerate(SBOX):
    INV_SBOX[_s] = _i
RCON = [0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36]
MUL = {c: [gmul(x, c) for x in range(256)] for c in (2, 3, 9, 11, 13, 14)} # таблицы
для MixColumns

# --- Раунд
-----
def expand_key(key):
    """Расширение ключа (FIPS-197, 5.2). Возвращает список раундовых ключей по 16
    байт."""
    nk = len(key) // 4
    if len(key) % 4 or nk not in (4, 6, 8):
        raise ValueError("ключ AES должен быть длиной 16, 24 или 32 байта")
    nr = nk + 6
    words = [list(key[4 * i:4 * i + 4]) for i in range(nk)]
    for i in range(nk, 4 * (nr + 1)):
        t = list(words[i - 1])
        if i % nk == 0:
            t = [SBOX[b] for b in t[1:] + t[:1]]      # RotWord + SubWord
            t[0] ^= RCON[i // nk - 1]
        elif nk > 6 and i % nk == 4:
            t = [SBOX[b] for b in t]
        words.append([words[i - nk][j] ^ t[j] for j in range(4)])
    return [sum(words[4 * r:4 * r + 4], []) for r in range(nr + 1)]

def _add_round_key(state, key):
    return [a ^ b for a, b in zip(state, key)]

def _sub_bytes(state, box):
    return [box[b] for b in state]

def _shift_rows(state):
    """Строка r сдвигается влево на r позиций: s'[r][c] = s[r][(c + r) mod 4]."""
    return [state[(i + 4 * (i % 4)) % 16] for i in range(16)]

def _inv_shift_rows(state):
    return [state[(i - 4 * (i % 4)) % 16] for i in range(16)]

def _mix_columns(state, inverse=False):
    """Умножение каждого столбца на фиксированный многочлен над GF(2^8) (FIPS-197,
    5.1.3 / 5.3.3)."""
    m2, m3, m9, m11, m13, m14 = (MUL[c] for c in (2, 3, 9, 11, 13, 14))
    out = []
    for c in range(4):
        a0, a1, a2, a3 = state[4 * c:4 * c + 4]
        if not inverse:
            out += [m2[a0] ^ m3[a1] ^ a2 ^ a3,
                    a0 ^ m2[a1] ^ m3[a2] ^ a3,
                    a0 ^ a1 ^ m2[a2] ^ m3[a3],

```

```

        m3[a0] ^ a1 ^ a2 ^ m2[a3]]
    else:
        out += [m14[a0] ^ m11[a1] ^ m13[a2] ^ m9[a3],
                m9[a0] ^ m14[a1] ^ m11[a2] ^ m13[a3],
                m13[a0] ^ m9[a1] ^ m14[a2] ^ m11[a3],
                m11[a0] ^ m13[a1] ^ m9[a2] ^ m14[a3]]
    return out

# --- Блок
-----
def encrypt_block(round_keys, block):
    """Шифрование одного 16-байтового блока (FIPS-197, 5.1)."""
    state = _add_round_key(list(block), round_keys[0])
    for key in round_keys[1:-1]:
        state = _add_round_key(_mix_columns(_shift_rows(_sub_bytes(state, SBOX))), key)
    state = _add_round_key(_shift_rows(_sub_bytes(state, SBOX)), round_keys[-1])
    return bytes(state)

def decrypt_block(round_keys, block):
    """Расшифрование одного блока обратным шифром (FIPS-197, 5.3)."""
    state = _add_round_key(list(block), round_keys[-1])
    for key in reversed(round_keys[1:-1]):
        state = _mix_columns(_add_round_key(_sub_bytes(_inv_shift_rows(state),
                INV_SBOX), key), inverse=True)
    state = _add_round_key(_sub_bytes(_inv_shift_rows(state), INV_SBOX), round_keys[0])
    return bytes(state)

# --- Режим CBC с дополнением PKCS#7
-----
def pad(data):
    n = BLOCK - len(data) % BLOCK
    return bytes(data) + bytes([n]) * n

def unpad(data):
    if not data or len(data) % BLOCK:
        raise ValueError("длина данных не кратна размеру блока")
    n = data[-1]
    if not 1 <= n <= BLOCK or data[-n:] != bytes([n]) * n:
        raise ValueError("неверное дополнение PKCS#7")
    return data[:-n]

def encrypt_cbc(key, iv, data):
    """C_i = E(P_i xor C_{i-1}), C_0 = iv. Возвращает шифртекст (длина кратна 16)."""
    if len(iv) != BLOCK:
        raise ValueError("вектор инициализации должен быть 16 байт")
    round_keys = expand_key(key)
    prev, out = bytes(iv), []
    padded = pad(data)
    for i in range(0, len(padded), BLOCK):
        block = bytes(a ^ b for a, b in zip(padded[i:i + BLOCK], prev))
        prev = encrypt_block(round_keys, block)
        out.append(prev)
    return b"".join(out)

def decrypt_cbc(key, iv, data):
    """P_i = D(C_i) xor C_{i-1}. Снимает дополнение; ValueError при повреждённых
    данных."""
    if len(iv) != BLOCK:
        raise ValueError("вектор инициализации должен быть 16 байт")
    if not data or len(data) % BLOCK:
        raise ValueError("длина шифртекста не кратна размеру блока")
    round_keys = expand_key(key)
    prev, out = bytes(iv), []
    for i in range(0, len(data), BLOCK):

```

```

        block = data[i:i + BLOCK]
        out.append(bytes(a ^ b for a, b in zip(decrypt_block(round_keys, block),
            prev)))
        prev = block
    return unpad(b"".join(out))

# --- Самопроверка
-----
FIPS_VECTORS = [    # FIPS-197, приложение C: (ключ, открытый текст, шифртекст)
    ("000102030405060708090a0b0c0d0e0f", "00112233445566778899aabbccddeeff",
     "69c4e0d86a7b0430d8cdb78070b4c55a"),
    ("000102030405060708090a0b0c0d0e0f1011121314151617",
     "00112233445566778899aabbccddeeff", "dda97ca4864cdfe06eaf70a0ec0d7191"),
    ("000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f",
     "00112233445566778899aabbccddeeff", "8ea2b7ca516745bfeafc49904b496089"),
]

def self_test(verbose=True):
    """Тестовые векторы FIPS-197 (AES-128/192/256) и обратимость CBC на случайных
    данных."""
    ok = True
    assert SBOX[0x00] == 0x63 and SBOX[0x01] == 0x7C and SBOX[0x53] == 0xED
    for key, plain, cipher in FIPS_VECTORS:
        rk = expand_key(bytes.fromhex(key))
        got = encrypt_block(rk, bytes.fromhex(plain)).hex()
        back = decrypt_block(rk, bytes.fromhex(cipher)).hex()
        good = got == cipher and back == plain
        ok &= good
    if verbose:
        print(f"AES-{len(key) * 4}: E(текст) = {got} {'совпал' if good else 'НЕ СОВПАЛ'} с FIPS-197; D(шифртекст) = {back}")
    for length in (0, 1, 15, 16, 17, 100, 1000):
        key, iv, data = os.urandom(32), os.urandom(16), os.urandom(length)
        ct = encrypt_cbc(key, iv, data)
        good = decrypt_cbc(key, iv, ct) == data and len(ct) == (length // BLOCK + 1) *
            BLOCK
        ok &= good
    if verbose:
        print(f"CBC, {length:4d} байт: шифртекст {len(ct):4d} байт, расшифрование {'верно' if good else 'НЕВЕРНО'}")
    if verbose:
        print("Итог:", "все проверки пройдены" if ok else "ЕСТЬ ОШИБКИ")
    return ok

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(errors="replace")
    sys.exit(0 if self_test() else 1)

```

Рисунок П1.3 – Листинг модуля *aes.py*